Laboratorio de Reingeniería: Sistema de Cuentas Bancarias

Ingeniería de Software II
Universidad Industrial de Santander (UIS)
Profesor: Urbano Gomez
Estudiantes:
Juan Esteban Fiagá Suárez
German Yesid Vanegas

30 de octubre de 2026

1. Descripción del caso

Se debe aplicar reingeniería a un programa bancario de la década de 1970: una sola sede, sin persistencia de datos y un único cliente por ejecución. El sistema original solo funciona para cuentas corrientes. La reingeniería busca soportar también cuentas de ahorros y mejorar la interfaz y el historial de movimientos.

2. Análisis del sistema actual (ingeniería inversa)

Se revisó el código entregado (`CuentaCorriente`, `CuentaAhorros` y `frmTransacciones`) y se encontraron estos puntos:

- `CuentaAhorros` hereda de `CuentaCorriente`; no existe una clase base común.
- El saldo y los métodos `depositar`, `retirar` y `getbalance` son estáticos, por lo que el saldo es global y no pertenece a un objeto cuenta.
- El interés de ahorros está mal implementado: usa tasa de 2 %, dividida entre 12, dentro de un getter que modifica el saldo, y el formulario nunca lo invoca.
- `retirar()` solo verifica que el saldo no sea cero; la validación de fondos está en el formulario y usa `saldo >retiro`, lo que impide retirar exactamente todo el saldo.
- La lógica de negocio y el historial (texto en un `JTextArea`) están en la vista. No existe una clase `Transaccion`.
- No hay validación de entrada: un valor no numérico lanza excepción.

3. Listado de requerimientos funcionales

3.1. Actores

- **Cliente:** único usuario del sistema. Selecciona el tipo de cuenta y registra movimientos.

3.2. Requerimientos

ID	Requerimiento	Entradas	Salidas
RF-01	Al iniciar, el sistema debe permitir elegir el tipo de cuenta mediante un ComboBox con las opciones Corriente y Ahorros. La elección se hace solo al iniciar y no puede cambiarse durante la ejecución.	Tipo de cuenta	Cuenta creada con saldo inicial 0 e interfaz de movimientos habilitada
RF-02	El sistema debe ofrecer una interfaz unificada con un campo de texto para el valor, un ComboBox de tipo de transacción (Consignación / Retiro) y un único botón "Registrar movimiento".	Valor, tipo de transacción	Transacción registrada o mensaje de error
RF-03	El sistema debe validar que el valor ingresado sea numérico y mayor que cero.	Valor ingresado	Aceptación o mensaje de error
RF-04	En una consignación, el sistema debe sumar el valor al saldo.	Valor	Nuevo saldo
RF-05	En un retiro, el sistema debe restar el valor al saldo solo si el valor es menor o igual al saldo (se permite retirar todo el saldo). Si es mayor, rechaza el movimiento, deja el saldo igual y registra la observación "Fondos insuficientes".	Valor	Nuevo saldo, o rechazo con observación
RF-06	En cuentas de Ahorros, tras cada transacción exitosa (consignación o retiro) el sistema debe calcular un interés del 1 % sobre el saldo resultante y sumarlo al saldo. El interés es acumulativo: cada cálculo parte de un saldo que ya incluye los intereses anteriores.	Saldo tras la transacción	Saldo final con interés
RF-07	En cuentas Corrientes, el sistema no calcula ningún interés.	N/A	Saldo sin modificación adicional
RF-08	El sistema debe reemplazar el balance simple por una tabla de historial con las columnas: # Transacción, Tipo, Valor, Saldo Resultante y Observaciones.	N/A	Tabla actualizada con cada movimiento
RF-09	Cada movimiento, incluso el rechazado, debe agregarse al historial con número consecutivo y su observación.	Datos de la transacción	Nueva fila en la tabla
RF-10	El sistema no debe almacenar datos de forma persistente: al cerrar la aplicación se pierden el saldo y el historial.	N/A	N/A

3.3. Criterio de aceptación (ejemplo de RF-08)

Ejemplo para una cuenta Corriente, tomado del enunciado:

# Transacción	Tipo	Valor	Saldo Resultante	Observaciones
1	Consignación	100	100	
2	Retiro	50	50	
3	Retiro	60	50	Fondos insuficientes

3.4. Supuestos

1. El saldo inicial de ambas cuentas es 0, porque el enunciado no indica otro valor.
2. La cuenta Corriente no permite sobregiro, porque el enunciado no lo menciona.
3. Si un retiro es rechazado, no se aplica el 1 % de interés, porque no hubo transacción efectiva.
4. El interés se refleja en el Saldo Resultante y se anota en Observaciones (por ejemplo, "Interés 1 % aplicado"), sin crear una fila aparte.

4. Diagrama de clases

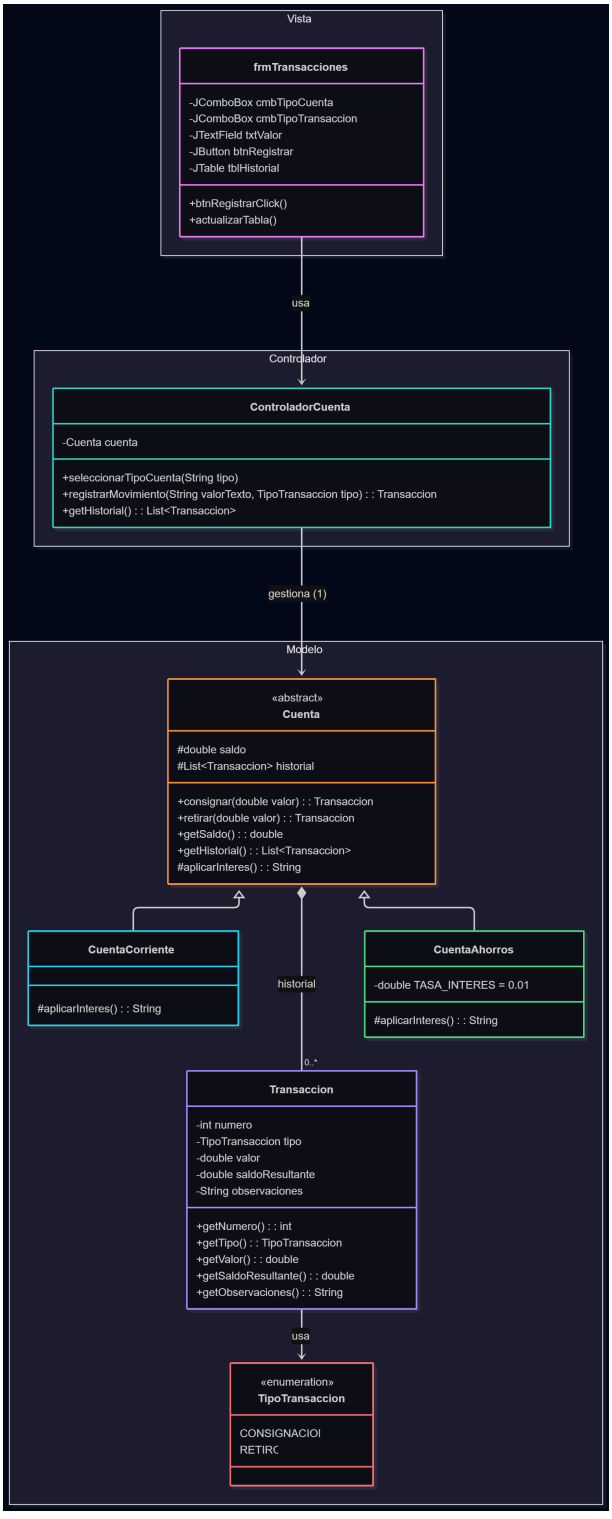


Figura 1: Diagrama de clases de la solución

4.1. Descripción de las clases

Clase	Responsabilidad	Atributos	Métodos
Cuenta (abstracta)	Lógica común de consignar, retirar y validar fondos; guarda el historial	`saldo`, `historial`	`consignar()`, `retirar()`, `getSaldo()`, `getHistorial()`, `aplicarInteres()` abstracto
CuentaCorriente	Cuenta sin intereses	(hereda)	`aplicarInteres()`: no hace nada
CuentaAhorros	Cuenta que paga 1 % tras cada transacción	`TASA_INTERES = 0.01`	`aplicarInteres()`: calcula y suma el 1 %
Transaccion	Representa un movimiento del historial	`numero`, `tipo`, `valor`, `saldoResultante`, `observaciones`	Getters
TipoTransaccion (enum)	Tipos de movimiento	`CONSIGNACION`, `RETIRO`	N/A
ControladorCuenta	Conecta la vista con el dominio y valida la entrada (RF-03)	`cuenta`	`seleccionarTipoCuenta()`, `registrarMovimiento()`, `getHistorial()`
frmTransacciones	Interfaz gráfica sin lógica de negocio	`cmbTipoCuenta`, `cmbTipoTransaccion`, `txtValor`, `btnRegistrar`, `tblHistorial`	`btnRegistrarClick()`, `actualizarTabla()`

4.2. Relaciones

- `CuentaCorriente` y `CuentaAhorros` heredan de `Cuenta`.
- `Cuenta` se compone de cero o más `Transaccion` (historial).
- `Transaccion` usa `TipoTransaccion`.
- `ControladorCuenta` gestiona una `Cuenta`.
- `frmTransacciones` usa `ControladorCuenta`.

4.3. Decisiones de diseño

- `Cuenta` es la clase base y elimina el uso de `static`, de modo que el saldo pertenece a cada objeto.
- `aplicarInteres()` es el único punto donde difieren las cuentas (polimorfismo).
- La validación de fondos pasa a la clase de dominio y usa `valor` $\leq$ `saldo`.
- La lógica sale de la vista y queda en el controlador y el dominio.

5. Diagrama de Actividades

El siguiente diagrama de actividades modela el flujo de ejecución del sistema basándose en los requerimientos funcionales establecidos. Representa la interacción del cliente desde la configuración inicial, pasando por las validaciones de entrada, hasta el registro en el historial y la finalización de la ejecución. El flujo ilustra además la separación de responsabilidades entre la Vista, el Controlador y el Dominio.

5.1. Descripción del Flujo

- **Configuración Inicial:** Al iniciar, el cliente debe seleccionar el tipo de cuenta (Corriente o Ahorros). Según el RF-01, esta es una decisión única que condiciona el resto de la ejecución

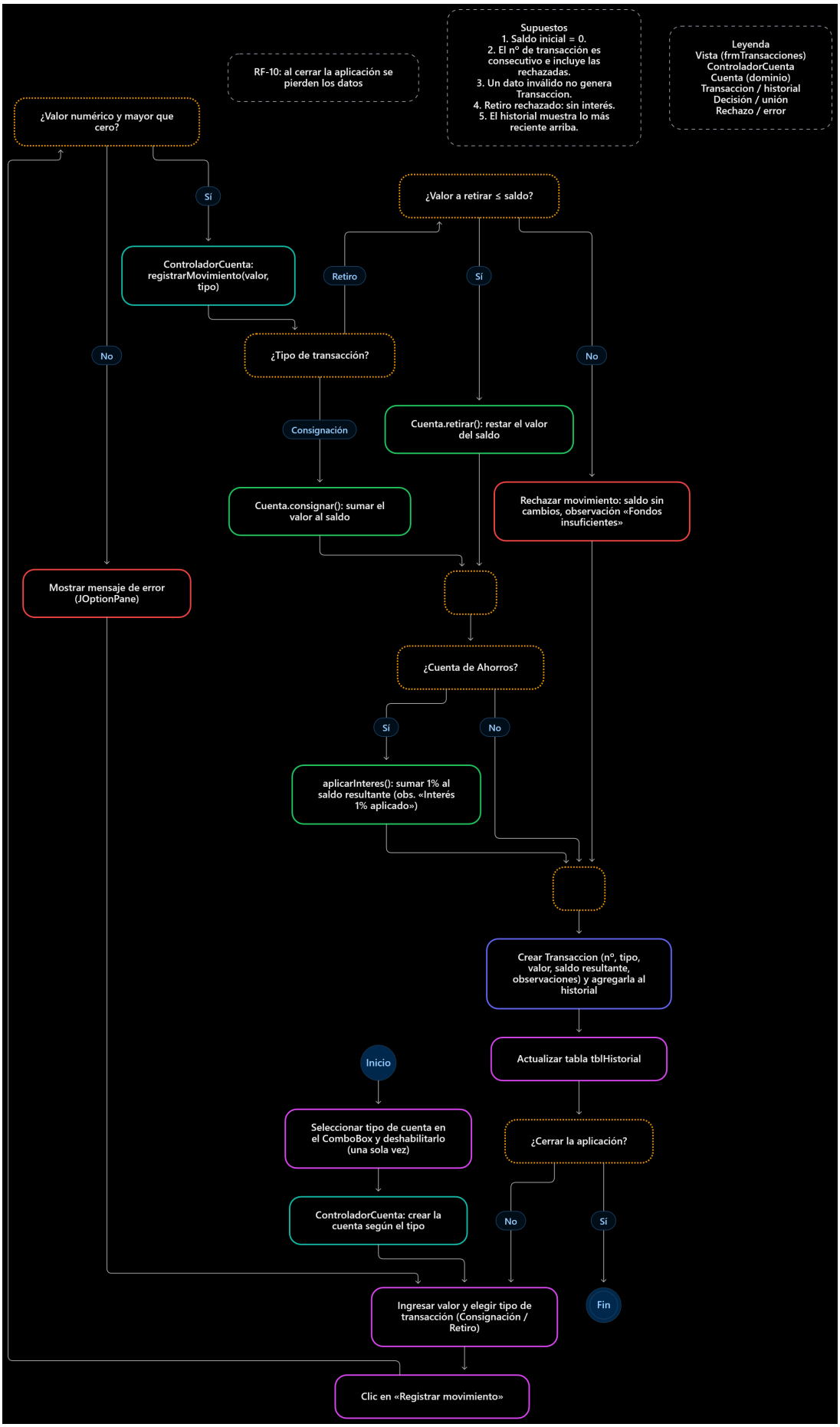


Figura 2: Flujo de actividades del sistema de transacciones bancarias.

y no puede alterarse posteriormente.

- **Validación de Entradas (RF-03):** Antes de procesar cualquier movimiento, el sistema verifica, a través del controlador, que el valor ingresado sea numérico y mayor que cero. Si la validación falla, se notifica el error mediante la interfaz gráfica y se detiene el flujo de esa transacción, retornando a la espera de un nuevo ingreso.
- **Lógica de Transacciones (Dominio):**
  - **Consignación (RF-04):** Suma directamente el monto al saldo disponible.
  - **Retiro (RF-05):** Incorpora una validación de fondos. Si el monto supera el saldo, el movimiento es rechazado, conservando el saldo intacto e ingresando la observación "Fondos insuficientes" en el historial.
- **Aplicación de Intereses (RF-06 y RF-07):** El flujo diverge según el tipo de cuenta seleccionado al inicio. Si es una Cuenta de Ahorros y el movimiento fue exitoso, el sistema calcula automáticamente un 1 % sobre el saldo resultante y lo suma al acumulado (registrando la observación "Interés 1 % aplicado"). Las Cuentas Corrientes, o bien las transacciones que resultaron en rechazo, omiten este paso de forma directa.
- **Registro (RF-08 y RF-10):** Toda iteración finaliza con la instanciación de un objeto *Transaccion* que se refleja inmediatamente en la tabla de historial de la vista. Al terminar la ejecución y cerrar la aplicación, el flujo concluye advirtiendo la pérdida de los datos temporales (saldo e historial en memoria).